__init__.py — technical

`src/moe_pruning/models/__init__.py · 139 lines · moe_pruning.models.__init__`

Purpose

Plugin registry keyed by `config.model_type`.

Step by step

Classes and functions in source order. Signatures, line ranges, raised exceptions and settings are extracted from the code; numbered steps describe what the code does.

`class ModelPlugin`

lines 64-76 · dataclass

Adapter, task and HF auto class of one model family.

Field	Type	Default
`name`	str	
`model_types`	tuple[str, ...]	
`adapter_factory`	Callable[[], 'MoEAdapter']	
`task_factory`	Callable[[], 'TaskObjective']	
`auto_model_class`	type	
`description`	str	''

`ModelPlugin.adapter`

```
def adapter(self) -> 'MoEAdapter'
```

lines 72-73

New adapter instance.

`ModelPlugin.task`

```
def task(self) -> 'TaskObjective'
```

lines 75-76

New task-objective instance.

`register_plugin`

```
def register_plugin(plugin: ModelPlugin) -> ModelPlugin
```

lines 83-92 · raises ValueError

Add a plugin; reject a second plugin for the same model type.

1. For each model type of the plugin: if another plugin (different name) already holds it, raise `ValueError`; otherwise store it. Re-registering the same plugin is allowed (idempotent imports).

`load_plugins`

```
def load_plugins(extra_modules: list[str] | None=None) -> None
```

lines 95-110

Import built-in plugins, entry-point plugins and MODEL_PLUGINS modules (once).

1. Collect extra modules from the argument and the `MODEL_PLUGINS` environment variable.
2. Once per process: import every sub-package of `moe_pruning.models` (built-ins register on import), then load entry points in group `moe_pruning.models` (a failing third-party plugin only logs a warning).
3. Import the extra modules (every call).

`available_plugins`

```
def available_plugins() -> dict[str, ModelPlugin]
```

lines 113-115

All registered plugins by model type.

`get_plugin`

```
def get_plugin(model_type: str) -> ModelPlugin
```

lines 118-127 · raises NotImplementedError

Plugin for a model type (clear error if unsupported).

1. `load_plugins()`, then dictionary lookup; unknown type -> `NotImplementedError` listing supported types.

`plugin_for_model`

```
def plugin_for_model(model) -> ModelPlugin
```

lines 130-132

Plugin of a loaded model.

`plugin_for_source`

```
def plugin_for_source(source: str, revision: str | None=None, cache_dir: str | None=None) -> ModelPlugin
```

lines 135-139

Plugin of a local model directory or Hub id (reads only config.json).

1. Read `config.json` via `AutoConfig` (no weights) and look up its model type.

Module constants

Name	Value
`ENTRY_POINT_GROUP`	'moe_pruning.models'
`PLUGIN_ENV_VAR`	'MODEL_PLUGINS'
`_LOADED`	False

Errors raised

Function	Exception	Message (f-string fields in braces)
`register_plugin`	ValueError	model_type {model_type} is already handled by plugin {existing.name}; cannot register {plugin.name}
`get_plugin`	NotImplementedError	No model plugin for model_type={model_type}. Supported: {supported}. See moe_pruning/models/__init__.py for how to add one (or set {PLUGIN_ENV_VAR}).